

Mantenimiento de código

Configuración base y código compartido

Documento de referencia técnica destinado al mantenimiento y la modificación de esta herramienta. No sustituye la lectura del código; señala los puntos de modificación y los aspectos que deben considerarse.

1. Descripción general

Código.gs y appscript.json forman la base común del proyecto. Código.gs define el menú del complemento, abre cada panel, fuerza la autorización inicial y concentra los datos y las funciones que utiliza más de una herramienta; appscript.json declara los servicios de Google que el proyecto tiene permiso de usar.

El Centro de Conocimiento tiene su propio archivo, Centro_conocimiento.gs, descrito en un documento aparte; este documento cubre únicamente lo que es común a todas las herramientas.

2. Dónde vive cada cosa

Elemento	Ubicación	Qué hace
Menú del complemento	Código.gs función onOpen()	Agrega las opciones del menú a la hoja de cálculo al abrirla.
Apertura de cada panel	Código.gs abrirRedactor(), abrirRedactorSap(), abrirTicketsPequeno()/abrirTicketsGrande()	Crean el diálogo modeless de cada herramienta (el del Centro de Conocimiento está en Centro_conocimiento.gs).
Autorización inicial	Código.gs función verificarAutorizacion()	Fuerza la pantalla de permisos de Google antes de abrir un panel.
Lista de hojas sin consultor	Código.gs constante HOJAS_SIN_CONSULTOR	Fuente única, expuesta al Derivador mediante getHojasSinConsultor().
Adjuntos automáticos de Portales	Código.gs constante PDF_ADJUNTOS	IDs de Drive de los PDF que se adjuntan a cada borrador del Redactor de Correos Portales.
Registro de tickets	Código.gs registrarEnSheet(), registrarEnFilaConsultor(), getSheet()	Utilizadas únicamente por el Derivador de Tickets.
Comparación de nombres	Código.gs nombresCoinciden(),	Utilizadas por el Derivador para

	normalizar()	emparejar consultores.
Creación de borradores de correo	Código.gs crearBorradorGmail(), crearBorradorGmailSap()	Una función por cada redactor, independientes entre sí.
Escape de HTML	Código.gs función esc()	Utilizada por los redactores al construir el cuerpo del correo.
Permisos y servicios	appsscript.json	Declara qué servicios de Google puede usar el proyecto (Gmail, Drive, Sheets, correo del usuario).

3. Configuración que se puede editar con seguridad

- **HOJAS_SIN_CONSULTOR (Código.gs)** — única lista de hojas con estructura de fila prellenada por consultor; la utiliza el Derivador de Tickets.
- **PDF_ADJUNTOS (Código.gs)** — IDs de Drive de los PDF adjuntos automáticos del Redactor de Correos Portales.
- **oauthScopes (appsscript.json)** — permisos que Google solicitará al usuario. Agregar un permiso nuevo obliga a una reautorización; retirar uno que ya no se utiliza reduce la pantalla de permisos que ve el usuario.
- **timeZone (appsscript.json)** — zona horaria utilizada para formatear fechas y horas en todo el proyecto.

4. Puntos de modificación frecuentes

Modificación	Ubicación en el código
Incorporación de una hoja nueva con estructura “sin consultor”	Constante HOJAS_SIN_CONSULTOR, en Código.gs (afecta directamente al Derivador)
Incorporación de un PDF nuevo para adjuntar en los correos de Portales	Constante PDF_ADJUNTOS, en Código.gs
Incorporación de una opción nueva en el menú del complemento	Función onOpen(), en Código.gs
Ampliación o reducción de los permisos que solicita el proyecto	Arreglo oauthScopes, en appsscript.json
Corrección de un caso de comparación de nombres que no coincide como se espera	Funciones nombresCoinciden() / normalizar(), en Código.gs

5. Advertencias e invariantes

Antes de modificar esta configuración deben considerarse los siguientes aspectos:

- HOJAS_SIN_CONSULTOR es fuente única: el Derivador la recibe siempre en tiempo de ejecución mediante getHojasSinConsultor(). No debe existir una copia local de esta lista en ningún archivo HTML.
- verificarAutorizacion() debe ejecutarse antes de abrir cualquier panel que utilice Gmail o Drive. Retirar esta llamada no impide que el panel se abra, pero traslada el error de permisos a un momento posterior y menos claro para el usuario.
- Cada herramienta abre su panel mediante HtmlService.createHtmlOutputFromFile(), y ese nombre de archivo debe coincidir exactamente, incluidas mayúsculas, con el nombre real del archivo en el proyecto de Apps Script.
- Los dos redactores de correo son independientes de manera intencional: cada uno cuenta con su propia función de servidor. No deben unificarse en una sola función compartida; esta separación responde a una decisión de diseño orientada a mantener la modularidad del proyecto.
- Retirar un permiso de oauthScopes sin verificar que ninguna función lo siga utilizando puede interrumpir esa función en tiempo de ejecución: el error solo se manifiesta al usarla, no al guardar el archivo.
- esc() debe aplicarse a cualquier dato proveniente del usuario que se inserte como HTML en el cuerpo de un correo, para evitar que un texto malformado altere el diseño o introduzca contenido no deseado.